DATA ENGINEER

Roles, Responsibilities & 60-Day Learning Roadmap

A Structured Transition Guide for any Database Administration Professionals

Prepared by: PMSQLDBA

Document Version 1.0 | July 2026

1. Data Engineer — Role Overview

A Data Engineer builds and maintains the infrastructure and pipelines that move data from source systems into forms that analysts, scientists, and applications can consume. Where a Database Administrator keeps a database healthy and performant, a Data Engineer designs the systems that feed data into and out of databases, warehouses, and data lakes at scale.

1.1 Position Snapshot

Attribute	Details
Primary Focus	Building and operating scalable data pipelines and architecture
Typical Reports To	Data Engineering Manager / Head of Data / Director of Analytics
Works Closely With	• Data Analysts • Data Scientists • Software / Application Engineers • BI Developers • ML Engineers
Core Deliverables	• Reliable ETL/ELT pipelines • Clean, query-ready datasets • Data warehouse / lake architecture • Orchestration and monitoring frameworks
Overlaps With DBA Role	SQL expertise, schema design, performance tuning, HA/DR awareness, compliance controls

2. Core Roles & Responsibilities

#	Responsibility Area	What It Involves
1	Data Pipeline Development (ETL/ELT)	• Design and build batch and streaming pipelines • Extract from databases, APIs, files, event streams • Transform (clean, join, aggregate, deduplicate) • Load into warehouses, lakes, or lakehouses
2	Data Architecture	• Design schemas for OLTP vs. OLAP workloads • Build star/snowflake schemas and data vault models • Choose warehouse vs. lake vs. lakehouse architecture • Partition, cluster, and index for scale
3	Data Quality & Governance	• Build validation and testing frameworks (data contracts) • Implement data lineage tracking • Handle PII / compliance requirements (SOX, PCI-DSS)
4	Orchestration & Automation	• Schedule and monitor pipeline jobs • Build retry logic, alerting, failure handling • Implement CI/CD for data pipelines

#	Responsibility Area	What It Involves
5	Performance & Cost Optimization	• Tune distributed query engines (Spark, Snowflake, Redshift, BigQuery) • Manage cloud compute / storage cost • Optimize file formats (Parquet, ORC, Avro) and partitioning
6	Cross-Functional Collaboration	• Expose clean, reliable datasets to Analysts / Scientists • Define data contracts with application engineering teams • Support ML Engineers with feature pipelines

3. DBA vs. Data Engineer — Skill Gap Analysis

This comparison maps your existing SQL Server / PostgreSQL DBA strengths against Data Engineer requirements to identify the true skill gap.

Area	DBA (Current)	Data Engineer (Target)	Gap to Close
SQL	Expert	Expert	None — direct advantage
Schema Design	OLTP-focused	OLTP + OLAP (dimensional modeling)	Learn star schema / dimensional modeling
Automation	PowerShell, dbatools	Python, Airflow, dbt	Learn Python + orchestration tools
Cloud	AWS/Azure VM, networking	Managed data services (S3, Redshift, Glue, ADF, Synapse)	Shift from infra layer to data-service layer
HA / Performance	Always On AG, wait stats, indexing	Distributed compute tuning (Spark, warehouse clusters)	Learn distributed systems fundamentals
Compliance	SOX / PCI-DSS	Data governance & lineage	Mostly transferable

NOTE: Your SQL depth, execution-plan intuition, and compliance background are direct, transferable advantages — the 60-day plan is sequenced to build on them before introducing new tools.

4. 60-Day Learning Roadmap

Assumes approximately 2–3 hours of focused study per day. Sequenced to build on existing SQL/cloud strengths first, then layer in Python, distributed processing, and orchestration.

4.1 Roadmap at a Glance

Phase	Days	Focus Area	Primary Milestone
Phase 1	1–10	Python for Data Engineering	API-to-database Python ingestion script
Phase 2	11–20	Data Modeling & Warehousing	Star schema built in Snowflake / Redshift
Phase 3	21–32	ETL/ELT Pipeline Tools	End-to-end pipeline orchestrated in Airflow
Phase 4	33–44	Distributed Processing & Streaming	PySpark job on cloud storage data
Phase 5	45–52	Data Quality & Governance	Automated tests + lineage diagram
Phase 6	53–60	Portfolio & Interview Readiness	Published portfolio project + cert prep

4.1 Phase 1 — Python for Data Engineering (Days 1–10)

Goal: Get functionally strong in Python for data manipulation — the foundation every modern DE tool sits on.

Days	Topics Covered	Tools / Concepts
1–3	• Python fundamentals • Data types, control flow, functions • List/dict comprehensions, file I/O • Virtual environments	Python 3.x, venv/pip
4–6	• DataFrames and transformations • Merging/joining datasets • GroupBy and aggregation • Handling nulls • Reading/writing CSV, Parquet, JSON	Pandas
7–8	• Working with REST APIs • Pagination, auth, rate limiting	requests library
9–10	• Python + SQL integration • Parameterized queries • Connection pooling	psycopg2, pyodbc, SQLAlchemy

BEST PRACTICE: Write a Python script that pulls data from a REST API and loads it into a Postgres/SQL Server table.

4.2 Phase 2 — Data Modeling & Warehousing (Days 11–20)

Goal: Translate existing OLTP schema expertise into OLAP/analytical modeling.

Days	Topics Covered	Tools / Concepts
11–13	• Dimensional modeling • Star vs. snowflake schema • Fact vs. dimension tables • Slowly Changing Dimensions (Types 1/2/3)	Dimensional modeling theory
14–16	• OLTP vs. OLAP concepts • Columnar storage • Warehouse indexing vs. AG indexing	Warehouse architecture concepts
17–18	• Deep dive on one cloud warehouse • Fast path: Snowflake (builds on SnowPro Core) • Alternative: Redshift (AWS-native)	Snowflake or Amazon Redshift
19–20	• File formats and compression • Parquet vs. ORC vs. Avro vs. CSV • Why columnar formats matter for analytics	Parquet, ORC, Avro

BEST PRACTICE: Design a star schema for a sample business domain (e.g., retail sales) and implement it in Snowflake or Redshift.

4.3 Phase 3 — ETL/ELT Pipeline Tools (Days 21–32)

Goal: Learn the actual tools used to build and orchestrate pipelines in production.

Days	Topics Covered	Tools / Concepts
21–23	• dbt models and materializations • ref() and macros • dbt tests	dbt (data build tool)
24–27	• DAGs, operators, sensors • Scheduling and dependencies • XComs, connections, hooks • Local Airflow via Docker	Apache Airflow
28–30	• Cloud-native orchestration • AWS: Glue + Step Functions • Azure: Data Factory	AWS Glue / Step Functions or Azure Data Factory
31–32	• Change Data Capture (CDC) • Debezium concepts • SQL Server CDC reframed as a pipeline source	Debezium, SQL Server CDC

BEST PRACTICE: Build an end-to-end pipeline: source DB → Python extraction → S3/Blob staging → dbt transformation → warehouse load, orchestrated in Airflow.

4.4 Phase 4 — Distributed Processing & Streaming (Days 33–44)

Goal: Understand big-data processing beyond single-server SQL.

Days	Topics Covered	Tools / Concepts
33–36	• Apache Spark fundamentals • RDDs vs. DataFrames • Transformations vs. actions • Partitioning and explain() plans	PySpark
37–39	• Spark on the cloud • Cluster sizing • Shuffles and data skew	Databricks or AWS EMR / Glue Spark
40–42	• Streaming fundamentals • Topics, partitions, consumer groups	Kafka, Kinesis, or Event Hubs
43–44	• Batch vs. streaming tradeoffs • Lambda vs. Kappa architecture	Architecture patterns (conceptual)

BEST PRACTICE: Run a PySpark job that reads a large dataset from cloud storage, transforms it, and writes it back partitioned.

4.5 Phase 5 — Data Quality & Governance (Days 45–52)

Goal: Apply the compliance background as a genuine differentiator.

Days	Topics Covered	Tools / Concepts
45–47	• Automated data quality checks • dbt tests • Great Expectations or Soda	Great Expectations / Soda
48–49	• Data lineage and cataloging • AWS Glue Data Catalog • Azure Purview • OpenLineage (open-source)	Glue Data Catalog, Purview, OpenLineage
50–52	• Apply SOX/PCI-DSS lens to pipelines • Access logging, masking, retention controls • Strong interview talking points	Governance & compliance mapping

BEST PRACTICE: Add data quality tests to the Phase 3 pipeline and document a data lineage diagram.

4.6 Phase 6 — Portfolio & Interview Readiness (Days 53–60)

Goal: Package everything into a demonstrable, hire able portfolio.

Days	Topics Covered	Tools / Concepts
53–55	• Build one polished end-to-end portfolio project • Reuse AG / AWS networking knowledge • Example: on-prem SQL Server → CDC → S3 → dbt → Redshift, orchestrated with Airflow	Integration of Phases 1–5
56–57	• Document the architecture • Clean GitHub README • Architecture diagram	GitHub, architecture documentation
58–59	• Cloud certification prep • AWS Data Engineer Associate or Azure DP-203	AWS / Azure certification guides
60	• Mock interview prep • Data modeling whiteboard questions • SQL optimization questions • Pipeline design questions	Interview practice

BEST PRACTICE: Publish one polished portfolio project over your personal GITHUB account with documentation and complete cert-readiness self-assessment.

5. Suggested Target Stack

Recommended stack given existing AWS, Azure, SQL Server, PostgreSQL, and Snowflake exposure.

Layer	Recommended Choice	Why
Language	Python + SQL	Universal DE language; SQL is already a strength
Warehouse	Snowflake (or Redshift)	Fastest ROI given prior SnowPro Core study; Redshift if targeting AWS-native roles
Transformation	dbt	SQL-based — fastest tool to pick up given existing SQL depth
Orchestration	Apache Airflow	Most widely hired-for orchestration tool
Cloud Certification	AWS Data Engineer Associate or Azure DP-203	Both realistic within a 60–75 day window given existing cloud footprint

BEST PRACTICE: Reuse existing SOP-writing discipline throughout this roadmap — documenting each milestone project the same way client-facing SOPs are documented turns the 60-day roadmap itself into a sellable knowledge product.